End of Day Report

Name	Atibali Saiyed
Date	11 November 2026
Topic	JavaScript (ES6+) — Core Concepts & Asynchronous Patterns
Code Repository	https://github.com/Atibali/Gateway-Training/tree/main/Week2/Day3

Part-1 JavaScript (ES6+)

1. Variables and scope (`let`, `const`, `var`) — `var` is function-scoped and hoisted with an initial value of `undefined`, while `let` and `const` are block-scoped and remain in the Temporal Dead Zone until their declarations are evaluated. `const` prevents reassignment but does not prevent mutation of an object. The recommended practice is to default to `const`, use `let` when reassignment is required, and avoid `var`.

2. Types — primitives vs objects, truthy/falsy, `==` vs `===` — Primitive values (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, and `bigint`) are compared by value, while objects and arrays are compared by reference. Reviewed the eight falsy values and learned that empty arrays (`[]`) and objects (`{}`) are truthy. The `==` operator performs type coercion, whereas `===` compares without coercion and is generally the safer choice.

3. Functions — declarations, expressions, arrow functions, default/rest/spread, closures, callbacks — Learned the difference between hoisted function declarations and function expressions, and that arrow functions inherit `this` from their enclosing scope. Practised default parameters, rest syntax (`...args`) to collect arguments, and spread syntax (`...arr`) to expand iterables. Understood closures as functions that retain access to their lexical scope, which can support data privacy, and callbacks as functions passed to be executed later.

4. Arrays — `map`, `filter`, `reduce`, `find`, `some`, `every`, `sort`, `forEach` — `map` and `filter` return new arrays without changing the original; `reduce` accumulates values into a single result; and `find`, `some`, and `every` return an element or Boolean result. Learned that `sort()` mutates the original array and sorts values as strings by default, so numeric sorting requires a comparator such as `(a, b) => a - b`. `forEach` is used for side effects and returns `undefined`.

5. Objects — literals, destructuring, spread, optional chaining, `Object.keys/values/entries` — Practised destructuring objects and arrays, including renaming, default values, and nested structures. Used spread syntax for shallow copying and merging. Optional chaining (`?.`) safely accesses nested properties when an intermediate value may be nullish. `Object.keys()`, `Object.values()`, and `Object.entries()` help iterate over objects, and can be combined with `Object.fromEntries()` to create objects from entries.

6. Classes — constructors, methods, getters/setters, inheritance (`extends`/`super`) — Learned that JavaScript classes provide syntax over the prototype-based inheritance model, and that `constructor()` runs when an instance is created with `new`. Practised getters and setters for controlled property access and validation, inheritance using `extends`, and calling `super()` before accessing `this` in a derived class constructor. Static methods belong to the class itself rather than its instances.

7. Modules — `import`/`export`, default vs named — Learned that a module can contain multiple named exports, imported using matching names inside `{}`, but only one default export. Also explored namespace imports (`import * as`), renaming imports with `as`, and dynamic `import()`, which returns a Promise. JavaScript modules run in strict mode and have their own scope.

8. Asynchronous JavaScript — event loop, callbacks, Promises, `async/await`, `try/catch`, `fetch` — Understood that JavaScript execution uses an event loop: queued microtasks (including Promise callbacks) are processed before the next macrotask, such as a `setTimeout` callback. Learned how Promises represent future results and use `.then()`, `.catch()`, and `.finally()`. Practised `async/await` for readable asynchronous code and `try/catch` for error handling. With `fetch`, an HTTP error does not automatically reject the Promise, so `response.ok` must be checked; `response.json()` is also asynchronous. Learned that `forEach()` does not wait for async callbacks; use `for...of` for sequential execution or `Promise.all()` with `map()` for parallel execution.

Key Takeaways

- Prefer `const` and `let` over `var`, and use `===` to avoid unintended type coercion.
- Closures support encapsulation and stateful functions without relying on global variables.
- Array methods are useful for data transformation; remember that `sort()` mutates the original array.
- Destructuring, spread syntax, and optional chaining reduce boilerplate and simplify safe property access.
- `async/await` with `try/catch` provides a clear way to handle Promises and asynchronous errors.
- Promise microtasks are processed before the next macrotask, such as a `setTimeout` callback.
- Always check `response.ok` when handling responses from `fetch`.